SDN-Lab4-Report

实验目的

1. 理解网络中网络故障出现的必然性
2. 理解网络验证工具 VeriFlow 的原理
3. 掌握 VeriFlow 检测网络故障的方法
4. 提高阅读工程代码、修改代码的能力

实验环境

- 操作系统：`x86/64 Arch Linux`
- Python 版本：3.9
- 控制器：Ryu (OpenFlow 1.0)
- 网络仿真：Mininet + Open vSwitch
- 验证工具：VeriFlow

热身

观察转发环路

实验现象

正常情况下 `ucla2 ping purdue` 可以连通。运行 `gen_loop.py` 后 ping 不通，100% packet loss。

```
▲ ziyu …/sdn-lab4  main !?  lab4 v3.9.25  ⊙ 17:04
» sudo ./simple.py
mininet> ucla2 ping purdue
PING 10.10.0.3 (10.10.0.3) 56(84) bytes of data.
64 bytes from 10.10.0.3: icmp_seq=1 ttl=64 time=163 ms
64 bytes from 10.10.0.3: icmp_seq=2 ttl=64 time=94.6 ms
64 bytes from 10.10.0.3: icmp_seq=3 ttl=64 time=94.4 ms
64 bytes from 10.10.0.3: icmp_seq=4 ttl=64 time=94.1 ms
64 bytes from 10.10.0.3: icmp_seq=5 ttl=64 time=94.1 ms
64 bytes from 10.10.0.3: icmp_seq=6 ttl=64 time=94.2 ms
64 bytes from 10.10.0.3: icmp_seq=7 ttl=64 time=94.1 ms
64 bytes from 10.10.0.3: icmp_seq=8 ttl=64 time=94.2 ms
64 bytes from 10.10.0.3: icmp_seq=9 ttl=64 time=94.1 ms
64 bytes from 10.10.0.3: icmp_seq=10 ttl=64 time=94.1 ms
64 bytes from 10.10.0.3: icmp_seq=11 ttl=64 time=94.1 ms
64 bytes from 10.10.0.3: icmp_seq=12 ttl=64 time=94.4 ms
64 bytes from 10.10.0.3: icmp_seq=13 ttl=64 time=94.5 ms
64 bytes from 10.10.0.3: icmp_seq=14 ttl=64 time=94.5 ms
64 bytes from 10.10.0.3: icmp_seq=15 ttl=64 time=94.5 ms
^C
--- 10.10.0.3 ping statistics ---
15 packets transmitted, 15 received, 0% packet loss, time 14014ms
rtt min/avg/max/mdev = 94.115/98.866/162.991/17.138 ms
```

（执行`gen_loop`后）

```
mininet> ucla2 ping purdue
PING 10.10.0.3 (10.10.0.3) 56(84) bytes of data.
^C
--- 10.10.0.3 ping statistics ---
8 packets transmitted, 0 received, 100% packet loss, time 7181ms
```

使用 `sudo ovs-ofctl dump-flows s3` 查看流表，发现 priority=10 规则的 `n_packets` 快速增长。

```
» sudo ovs-ofctl dump-flows s3
 cookie=0x0, duration=33.989s, table=0, n_packets=1891, n_bytes=185318, priority=10,ip,nw_dst=10.10.0.0/16 acti
ons=output:"s3-eth4"
 cookie=0x0, duration=207.679s, table=0, n_packets=15, n_bytes=1470, priority=5,ip,nw_src=10.10.0.4,nw_dst=10.1
0.0.3 actions=output:"s3-eth3"
 cookie=0x0, duration=207.609s, table=0, n_packets=15, n_bytes=1470, priority=5,ip,nw_src=10.10.0.3,nw_dst=10.1
0.0.4 actions=output:"s3-eth2"
 cookie=0x0, duration=320.224s, table=0, n_packets=14, n_bytes=1172, priority=0 actions=CONTROLLER:65509
```

（dump-flows 中 n_packets 异常）

原因分析

`gen_loop.py` 向 6 台交换机注入了 priority=10 的流表规则，匹配 `nw_dst=10.10.0.0/16`，构造了一条环形转发路径：

```
illinois(3) → wisconsin(4) → usc2(7) → usc1(2) → ucla1(1) → ucla2(6) → illinois(3) → ...
```

由于 `priority=10` 大于正常规则的 `priority=5`，所有目的地为 10.10.0.0/16 的数据包都被截获进入环路，在环中无限循环直到 TTL 耗尽。流表的 `n_packets` 快速增长正是数据包在环路中反复经过同一交换机的证据。

使用 VeriFlow 检测环路

实验现象

部署 VeriFlow 代理后，注入环路规则时，`veriflow.log` 中出现 LOOP 报告。

```
» cat veriflow.log

[VeriFlow::traverseForwardingGraph] Found a LOOP for the following packet class at node 20.0.0.006
.6.
[VeriFlow::traverseForwardingGraph] PacketClass: [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 2
81474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff)
, nw_src (168427524-10.10.0.4, 168427524-10.10.0.4), nw_dst (168427524-10.10.0.4, 168493055-10.10.
255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3
(2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (16
8427524, 168427524), Field 9 (168427524, 168493055), Field 10 (0, 255), Field 11 (0, 63), Field 12
 (0, 65535), Field 13 (0, 65535)

[VeriFlow::traverseForwardingGraph] Found a LOOP for the following packet class at node 20.0.0.006
.6.
[VeriFlow::traverseForwardingGraph] PacketClass: [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 2
81474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff)
, nw_src (168427525-10.10.0.5, 4294967295-255.255.255.255), nw_dst (168427520-10.10.0.0, 168493055
-10.10.255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), F
ield 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Fiel
d 8 (168427525, 4294967295), Field 9 (168427520, 168493055), Field 10 (0, 255), Field 11 (0, 63),
Field 12 (0, 65535), Field 13 (0, 65535)
```

原因分析

VeriFlow 部署在控制器（端口 1024）和交换机（端口 6633）之间，作为透明代理拦截所有 OpenFlow 消息。当 `gen_loop.py` 触发的 FLOW_MOD 消息经过 VeriFlow 时：

1. VeriFlow 解析该规则影响的等价类（EC）
2. 为每个 EC 构建转发图——查询所有交换机上匹配该 EC 的规则
3. 从规则所在交换机出发遍历转发图
4. 发现某节点已被访问 → 判定为环路

VeriFlow 能在规则下发的瞬间检测到问题，而不需要等待实际数据包产生故障。

修改 VeriFlow 源码

任务 1：输出每次影响 EC 的数量

修改内容

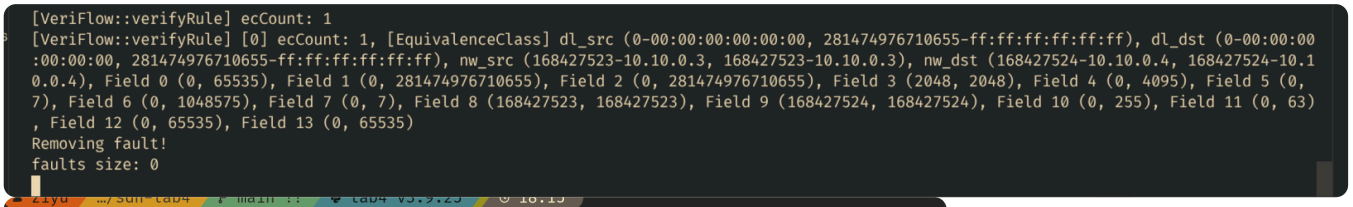
取消 `VeriFlow.cpp` 中 `verifyRule` 函数的所有有关 `ecCount` 的打印注释：

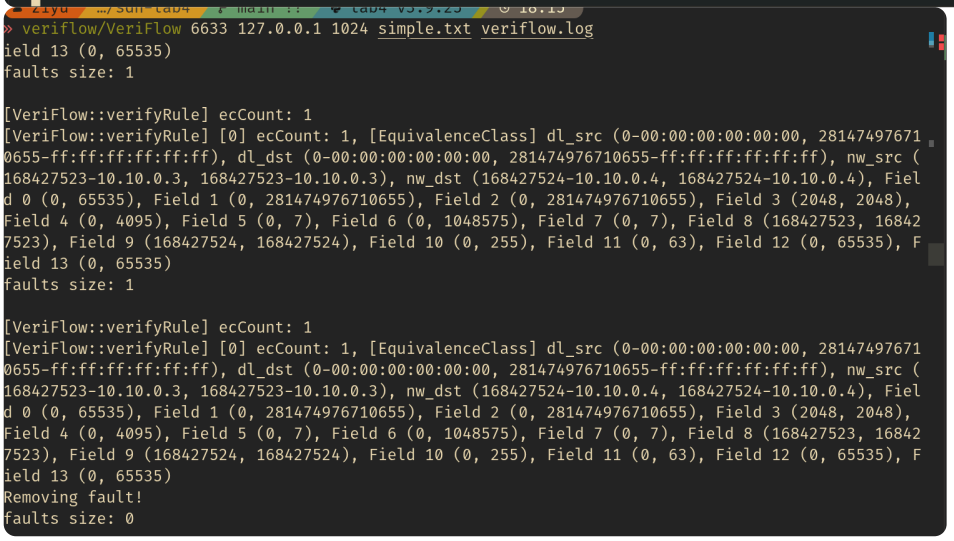
```C++
fprintf(stdout, "[VeriFlow::verifyRule] ecCount: %lu\n", ecCount);
```

原理说明

`ecCount` 表示一条新规则影响的等价类数量。EC 是 VeriFlow 将数据包空间按转发行为划分的最小单元。一条匹配范围广的规则（如 `10.10.0.0/16`）会影响更多 EC，验证开销也更大。

结果





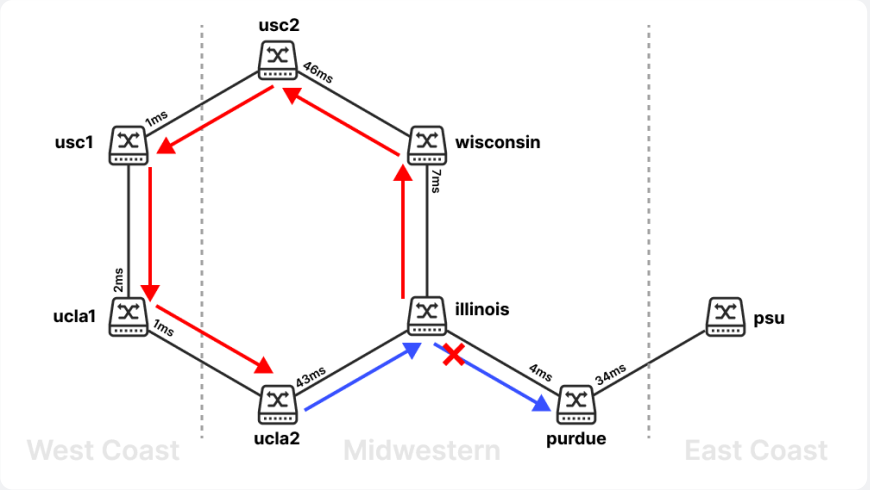
> 执行`gen_loop`前，可以看到已经注入了两条规则，每条规则都打印出`ecCount: 1`。每次经过解决`faluts size`是0。因为这样的一个点对点ping指令执行时，控制器安装的是精确匹配规则（`nw_src=10.10.0.3`，`nw_dst=10.10.0.4`，以及反过来的一条），所以只产生两个受影响的EC，也就是这对地址。

```
[VeriFlow::verifyRule] ecCount: 8
[VeriFlow::verifyRule] [0] ecCount: 8, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:
00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (0-0.0.0.0, 168427522-10.10.0.2), nw_dst (168427520-10.10.0.0, 168493055-10.10.255.
255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0,
7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (0, 168427522), Field 9 (168427520, 168493055), Field 10 (0, 255), Field 11 (0, 63), Fiel
d 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [1] ecCount: 8, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:
00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427523-10.10.0.3, 168427523-10.10.0.3), nw_dst (168427520-10.10.0.0, 168427523-
10.10.0.3), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field
5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168427523, 168427523), Field 9 (168427520, 168427523), Field 10 (0, 255), Field 11
 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [2] ecCount: 8, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:
00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427523-10.10.0.3, 168427523-10.10.0.3), nw_dst (168427524-10.10.0.4, 168427524-
10.10.0.4), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field
5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168427523, 168427523), Field 9 (168427524, 168427524), Field 10 (0, 255), Field 11
 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [3] ecCount: 8, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:
00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427523-10.10.0.3, 168427523-10.10.0.3), nw_dst (168427525-10.10.0.5, 168493055-
10.10.255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Fi
eld 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168427523, 168427523), Field 9 (168427525, 168493055), Field 10 (0, 255), Fiel
d 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [4] ecCount: 8, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:
00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427524-10.10.0.4, 168427524-10.10.0.4), nw_dst (168427520-10.10.0.0, 168427522-
10.10.0.2), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field
5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168427524, 168427524), Field 9 (168427520, 168427522), Field 10 (0, 255), Field 11
 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [5] ecCount: 8, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:
00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427524-10.10.0.4, 168427524-10.10.0.4), nw_dst (168427523-10.10.0.3, 168427523-
10.10.0.3), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field
5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168427524, 168427524), Field 9 (168427523, 168427523), Field 10 (0, 255), Field 11
 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [6] ecCount: 8, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:
00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427524-10.10.0.4, 168427524-10.10.0.4), nw_dst (168427524-10.10.0.4, 168493055-
10.10.255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Fi
eld 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168427524, 168427524), Field 9 (168427524, 168493055), Field 10 (0, 255), Fiel
d 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [7] ecCount: 8, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:
00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427525-10.10.0.5, 4294967295-255.255.255.255), nw_dst (168427520-10.10.0.0, 168
493055-10.10.255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 40
95), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168427525, 4294967295), Field 9 (168427520, 168493055), Field 10 (0, 25
5), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
faults size: 8
```

注入后，`ecCount`和`fautls size`都变成了8。因为此时它安装的是匹配`nw_dst=10.10.0.0/16`的宽泛规则。这个范围覆盖了整个子网。VeriFlow需要把新规则的匹配范围和已有规则（两条点对点）的匹配范围做交集切割。最终切出来8个EC。随后VeriFlow对这 8 个 EC 逐一做转发图遍历。`gen_loop.py` 创建的 loop 路径是`illinois→wisconsin→usc2→usc1→ucla1→ucla2→illinois`，匹配条件是`nw_dst=10.10.0.0/16`。这 8 个 EC 的 `nw_dst` 都落在`10.10.0.0/16`范围内，所以在那些 loop 涉及的交换机上，它们全部会命中这条 priority=10 的 loop。所以每个EC遍历的时候都检测到loop，因此这8个EC都报故障，所以有`faults size = 8`。

> ● 因为等价类的切割是由**已有规则的匹配边界**决定的。
>
> 在 gen_loop.py 注入之前，ucla2 ping purdue 已经让控制器安装了两条精确规则：
> - 规则 A: nw_src=10.10.0.3, nw_dst=10.10.0.4 (purdue→ucla2)
> - 规则 B: nw_src=10.10.0.4, nw_dst=10.10.0.3 (ucla2→purdue)
>
> 现在 gen_loop.py 插入一条 nw_dst=10.10.0.0/16 的规则（nw_src 是通配）。VeriFlow
> 要计算这条新规则影响了哪些等价类，就需要把新规则的范围和已有规则的范围做**交集切割**。
>
> **nw_src 维度的切割：**
>
> 已有规则在 nw_src 上有两个精确值：10.10.0.3 和 10.10.0.4。新规则的 nw_src 是全通配 [0, 255.255.255.255]。切割后：
> 1. [0.0.0.0, 10.10.0.2] — 在 10.10.0.3 之前
> 2. [10.10.0.3, 10.10.0.3] — 精确命中已有规则 A
> 3. [10.10.0.4, 10.10.0.4] — 精确命中已有规则 B
> 4. [10.10.0.5, 255.255.255.255] — 在 10.10.0.4 之后
>
> **nw_dst 维度的切割：**
>
> 已有规则在 nw_dst 上也有两个精确值：10.10.0.3 和 10.10.0.4。新规则的 nw_dst 是 [10.10.0.0, 10.10.255.255]。切割后：
> 1. [10.10.0.0, 10.10.0.2]
> 2. [10.10.0.3, 10.10.0.3]
> 3. [10.10.0.4, 10.10.0.4]
> 4. [10.10.0.5, 10.10.255.255]
>
> **两个维度做笛卡尔积：** 4 × 4 = 16 种组合。但不是所有组合都有效 — 只有那些在已有规则中实际存在对应转发行为差异的组合才会成为独立的 EC。最终 VeriFlow 计算出 8 个有意义的等价类。
>
> 你可以对照输出验证：
> - EC[0]: nw_src 是第 1 段（通配到 10.10.0.2），nw_dst 是整个 /16
> - EC[1-3]: nw_src = 10.10.0.3 (purdue)，nw_dst 被切成 3 段
> - EC[4-6]: nw_src = 10.10.0.4 (ucla2)，nw_dst 被切成 3 段
> - EC[7]: nw_src 是第 4 段（10.10.0.5 以后），nw_dst 是整个 /16
>
> **本质上：已有的精确规则在地址空间中插入了"切割点"，新的宽泛规则经过这些切割点时就被分成了多个等价类。**

经过AI的解答，明白了这8条规则如何产生。



上面的输出信息显示8个被改变的具体EC（也是8个出错的）信息如下：

EC	nw_src	nw_dst
[0]	0.0.0.0 ~ 10.10.0.2	10.10.0.0 ~ 10.10.255.255
[1]	10.10.0.3（purdue）	10.10.0.0 ~ 10.10.0.3
[2]	10.10.0.3（purdue）	10.10.0.4（ucla2）
[3]	10.10.0.3（purdue）	10.10.0.5 ~ 10.10.255.255
[4]	10.10.0.4（ucla2）	10.10.0.0 ~ 10.10.0.2
[5]	10.10.0.4（ucla2）	10.10.0.3（purdue）
[6]	10.10.0.4（ucla2）	10.10.0.4 ~ 10.10.255.255
[7]	10.10.0.5 ~ 255.255.255.255	10.10.0.0 ~ 10.10.255.255

任务 2：打印环路路径

修改内容

在 `traverseForwardingGraph` 函数中添加 `vector<string> path` 参数，记录遍历顺序，在检测到环路时打印有序路径。所以要修改 `VeriFlow.h` 中的声明和 `VeriFlow.cpp` 中的实际逻辑。

完整函数代码：

```C++
// 函数签名增加 vector<string> path 参数，如下：
// bool traverseForwardingGraph(const EquivalenceClass& packetClass, ForwardingGraph* graph, const string&
currentLocation, const string& lastHop, unordered_set < string > visited, vector< string > path, FILE*
fp);
bool VeriFlow::traverseForwardingGraph(const EquivalenceClass& packetClass, ForwardingGraph* graph, const
string& currentLocation, const string& lastHop, unordered_set< string > visited, vector< string > path,
FILE* fp)
{

   fprintf(fp, "traversing at node: %s\n", currentLocation.c_str());
   if(graph == NULL)
   {
      fprintf(fp, "\n");
      fprintf(fp, "[VeriFlow::traverseForwardingGraph] (graph == NULL) for the following packet class at
node %s.\n", currentLocation.c_str());
      fprintf(fp, "[VeriFlow::traverseForwardingGraph] PacketClass: %s\n", packetClass.toString().c_str());

      return true;
   }

   if(currentLocation.compare("") == 0)
   {
      return true;
   }
```

```
  if(visited.find(currentLocation) != visited.end()) //检测到环路
  {
    // Found a loop.
    fprintf(fp, "\n");
    fprintf(fp, "[VeriFlow::traverseForwardingGraph] Found a LOOP for the following packet class at node
%s.\n", currentLocation.c_str());
    fprintf(fp, "[VeriFlow::traverseForwardingGraph] PacketClass: %s\n", packetClass.toString().c_str());

    string pathStr = "";
    for(int i = 0; i < path.size(); i++) {
      pathStr += path[i] + " -> ";
    }
    pathStr += currentLocation;
    fprintf(fp, "%s\n", pathStr.c_str());
    for(unsigned int i = 0; i < faults.size(); i++) {
      if (packetClass.subsumes(faults[i])) {
        faults.erase(faults.begin() + i);
        i--;
      }
    }
    faults.push_back(packetClass);

    return false;
  }

  visited.insert(currentLocation);
  path.push_back(currentLocation); //把节点添加到path中

  if(graph->links.find(currentLocation) == graph->links.end())
  {
    // Found a black hole.
    fprintf(fp, "\n");
    fprintf(fp, "[VeriFlow::traverseForwardingGraph] Found a BLACK HOLE for the following packet class as
current location (%s) not found in the graph.\n", currentLocation.c_str());
    fprintf(fp, "[VeriFlow::traverseForwardingGraph] PacketClass: %s\n", packetClass.toString().c_str());
    for(unsigned int i = 0; i < faults.size(); i++) {
      if (packetClass.subsumes(faults[i])) {
        faults.erase(faults.begin() + i);
        i--;
      }
    }
    faults.push_back(packetClass);

    return false;
  }

  if(graph->links[currentLocation].empty() == true)
  {
    // Found a black hole.
    fprintf(fp, "\n");
    fprintf(fp, "[VeriFlow::traverseForwardingGraph] Found a BLACK HOLE for the following packet class as
there is no outgoing link at current location (%s).\n", currentLocation.c_str());
    fprintf(fp, "[VeriFlow::traverseForwardingGraph] PacketClass: %s\n", packetClass.toString().c_str());
    for(unsigned int i = 0; i < faults.size(); i++) {
      if (packetClass.subsumes(faults[i])) {
        faults.erase(faults.begin() + i);
        i--;
      }
    }
    faults.push_back(packetClass);

    return false;
  }
```

```
  graph->links[currentLocation].sort(compareForwardingLink);

  const list< ForwardingLink >& linkList = graph->links[currentLocation];
  list< ForwardingLink >::const_iterator itr = linkList.begin();
  // input_port as a filter
  if(lastHop.compare("NULL") == 0 || itr->rule.in_port == 0){
    // do nothing
  }
  else{
    while(itr != linkList.end()){
      string connected_hop = network.getNextHopIpAddress(currentLocation, itr->rule.in_port);
      if(connected_hop.compare(lastHop) == 0) break;
      itr++;
    }
  }

  if(itr == linkList.end()){
    // Found a black hole.
    fprintf(fp, "\n");
    fprintf(fp, "[VeriFlow::traverseForwardingGraph] Found a BLACK HOLE for the following packet class as
there is no outgoing link at current location (%s).\n", currentLocation.c_str());
    fprintf(fp, "[VeriFlow::traverseForwardingGraph] PacketClass: %s\n", packetClass.toString().c_str());
    for(unsigned int i = 0; i < faults.size(); i++) {
      if (packetClass.subsumes(faults[i])) {
        faults.erase(faults.begin() + i);
        i--;
      }
    }
    faults.push_back(packetClass);

    return false;
  }

  if(itr->isGateway == true)
  {
    // Destination reachable.
    // fprintf(fp, "[VeriFlow::traverseForwardingGraph] Destination reachable.\n");
    fprintf(fp, "\n");
    fprintf(fp, "[VeriFlow::traverseForwardingGraph] The following packet class reached destination at
node %s.\n", currentLocation.c_str());
    fprintf(fp, "[VeriFlow::traverseForwardingGraph] PacketClass: %s\n", packetClass.toString().c_str());
    for(unsigned int i = 0; i < faults.size(); i++) {
      if (packetClass.subsumes(faults[i])) {
        fprintf(stderr, "Removing fault!\n");
        faults.erase(faults.begin() + i);
        i--;
      }
    }
    return true;
  }
  else
  {
    // Move to the next location.
    // fprintf(fp, "[VeriFlow::traverseForwardingGraph] Moving to node %s.\n", itr-
>rule.nextHop.c_str());

    if(itr->rule.nextHop.compare("") == 0)
    {
      // This rule is a packet filter. It drops packets.
      /* fprintf(fp, "\n");
      fprintf(fp, "[VeriFlow::traverseForwardingGraph] The following packet class is dropped by a packet
filter at node %s.\n", currentLocation.c_str());
      fprintf(fp, "[VeriFlow::traverseForwardingGraph] PacketClass: %s\n",
packetClass.toString().c_str()); */
    }
```

```
    return this->traverseForwardingGraph(packetClass, graph, itr->rule.nextHop, currentLocation, visited,
path, fp);
  }
}
```

原理说明

原来的 `visited`（unordered_set）可以 O(1) 判断是否已访问，但不保留顺序。所以增加一个vector `path` 按顺序记录节点。**visited 检测环路，path 还原路径。**

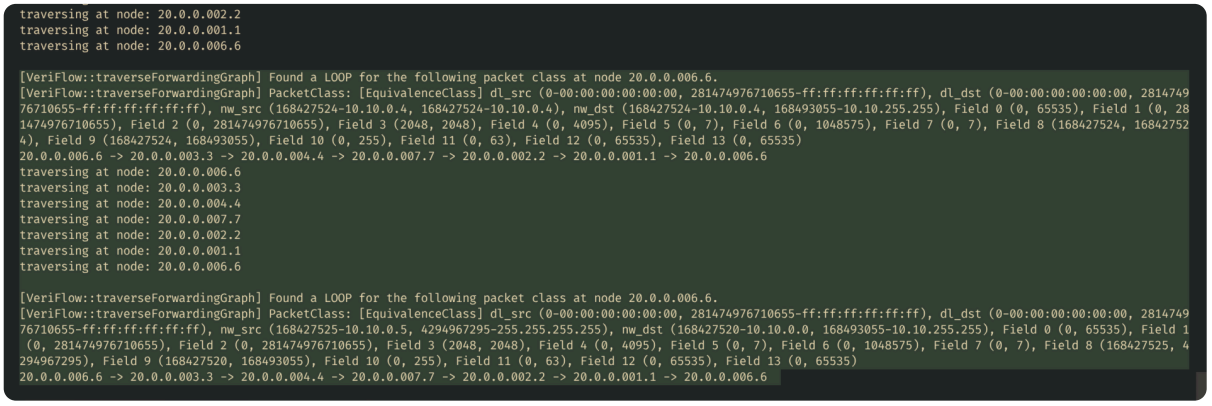
由于 visited 和 path 都是按值传递（递归时拷贝），回溯自动恢复，无需手动 pop。

结果

预计会出现如下的环路。

```
20.0.0.006.6 → 20.0.0.003.3 → 20.0.0.004.4 → 20.0.0.007.7 → 20.0.0.002.2 → 20.0.0.001.1 →
20.0.0.006.6
```

对应路径：`ucla2 → illinois → wisconsin → usc2 → usc1 → ucla1 → ucla2（回到起点形成环路）`



日志证明了这一点。

任务 3：打印 EC 五元组+MAC 信息

修改内容

在检测到 LOOP 时，自定义 PacketClass 输出格式，让信息更清晰，只展示源/目的MAC与TCP/IP的5元组，具体内容如下：

- 源与目的MAC；
- 源与目的IP；
- 协议号
- 源与目的端口

C++

```
fprintf(fp, "[VeriFlow::traverseForwardingGraph] PacketClass: [EquivalenceClass] " // 替换默认打印语句
  "dl_src (%s, %s), dl_dst (%s, %s), "
  "nw_src (%s, %s), nw_dst (%s, %s), "
  "nw_proto (%lu, %lu), tp_src (%lu, %lu), tp_dst (%lu, %lu)\n",
  ::getMacValueAsString(packetClass.lowerBound[DL_SRC]).c_str(),
  ::getMacValueAsString(packetClass.upperBound[DL_SRC]).c_str(),
  ::getMacValueAsString(packetClass.lowerBound[DL_DST]).c_str(),
  ::getMacValueAsString(packetClass.upperBound[DL_DST]).c_str(),
  ::getIpValueAsString(packetClass.lowerBound[NW_SRC]).c_str(),
  ::getIpValueAsString(packetClass.upperBound[NW_SRC]).c_str(),
  ::getIpValueAsString(packetClass.lowerBound[NW_DST]).c_str(),
  ::getIpValueAsString(packetClass.upperBound[NW_DST]).c_str(),
  packetClass.lowerBound[NW_PROTO], packetClass.upperBound[NW_PROTO],
  packetClass.lowerBound[TP_SRC], packetClass.upperBound[TP_SRC],
```

```
    packetClass.lowerBound[TP_DST], packetClass.upperBound[TP_DST]);
```

结果

```
[VeriFlow::traverseForwardingGraph] Found a LOOP for the following packet class at node 20.0.0.006.6.
[VeriFlow::traverseForwardingGraph] PacketClass: [EquivalenceClass] dl_src (00:00:00:00:00:00, ff:ff:ff:ff:ff:ff), dl_dst (00:00:00:00:00:00, ff:ff:ff:ff:ff:ff), nw_src
(10.10.0.4, 10.10.0.4), nw_dst (10.10.0.4, 10.10.255.255), nw_proto (0, 255), tp_src (0, 65535), tp_dst (0, 65535)
20.0.0.006.6 -> 20.0.0.003.3 -> 20.0.0.004.4 -> 20.0.0.007.7 -> 20.0.0.002.2 -> 20.0.0.001.1 -> 20.0.0.006.6
traversing at node: 20.0.0.006.6
traversing at node: 20.0.0.003.3
traversing at node: 20.0.0.004.4
traversing at node: 20.0.0.007.7
traversing at node: 20.0.0.002.2
traversing at node: 20.0.0.001.1
traversing at node: 20.0.0.006.6

[VeriFlow::traverseForwardingGraph] Found a LOOP for the following packet class at node 20.0.0.006.6.
[VeriFlow::traverseForwardingGraph] PacketClass: [EquivalenceClass] dl_src (00:00:00:00:00:00, ff:ff:ff:ff:ff:ff), dl_dst (00:00:00:00:00:00, ff:ff:ff:ff:ff:ff), nw_src
(10.10.0.5, 255.255.255.255), nw_dst (10.10.0.0, 10.10.255.255), nw_proto (0, 255), tp_src (0, 65535), tp_dst (0, 65535)
20.0.0.006.6 -> 20.0.0.003.3 -> 20.0.0.004.4 -> 20.0.0.007.7 -> 20.0.0.002.2 -> 20.0.0.001.1 -> 20.0.0.006.6
```

这下可以了。

修复 Fault 计算（必做题）

黑洞（black hole） 就是*数据包走到某个节点后，没有匹配的转发规则，包被丢弃了*。以上面的ping命令为例：执行ping命令后，控制器只为 `ucla1 ping illinois` 这对精确地址安装了规则，但 VeriFlow 验证时发现，有些**更宽泛的等价类**（比如 `nw_src=10.12.0.0/16 → nw_dst=10.10.0.0/16`）的数据包，在转发到某个交换机（`20.0.0.006.6` 即 usc1）时，usc1 上没有匹配的转发规则 — 包到这里就"消失"了。

实验现象

执行 `ucla1 ping illinois`（跨 AS）后，VeriFlow 报告黑洞。

```
[VeriFlow::traverseForwardingGraph] Found a BLACK HOLE for the following packet class as current location (20.0
.0.001.1) not found in the graph.
[VeriFlow::traverseForwardingGraph] PacketClass: [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 28147497671065
5-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427520-10.10.
0.0, 168460287-10.10.127.255), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field
 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7),
 Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168427520, 168460287), Field 9 (168558592, 168624127), Field 10
 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
traversing at node: 20.0.0.001.1

[VeriFlow::traverseForwardingGraph] The following packet class reached destination at node 20.0.0.001.1.
[VeriFlow::traverseForwardingGraph] PacketClass: [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 28147497671065
5-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427521-10.10.
0.1, 168427521-10.10.0.1), nw_dst (168558593-10.12.0.1, 168558593-10.12.0.1), Field 0 (0, 65535), Field 1 (0, 2
81474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6
```

> 这里的警告提示，VeriFlow 在验证 EC（nw_src=10.10.0.0~10.10.127.255, nw_dst=10.12.0.0~10.12.255.255）时，从 usc1（20.0.0.006.6）开始遍历转发图。但是 usc1 上**没有任何规则能匹配这个 EC 的数据包**。包到了 `usc1` 就没有下一跳，直接被丢弃了。这就是所谓 **“黑洞”**。

运行 `fix_path.py` 理论上修复了问题，但 `faults size` 始终不归零：

```
[VeriFlow::verifyRule] [6] ecCount: 11, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-
ff:ff:ff:ff:ff:ff), nw_src (168460288-10.10.128.0, 168558591-10.11.255.255), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 28147
4976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168460288, 168558591),
 Field 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [7] ecCount: 11, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-
ff:ff:ff:ff:ff:ff), nw_src (168558592-10.12.0.0, 168558592-10.12.0.0), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 2814749767171
0655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168558592, 168558592), Field
 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [8] ecCount: 11, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-
ff:ff:ff:ff:ff:ff), nw_src (168558593-10.12.0.1, 168558593-10.12.0.1), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 2814749767171
0655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168558593, 168558593), Field
 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [9] ecCount: 11, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-
ff:ff:ff:ff:ff:ff), nw_src (168558594-10.12.0.2, 168624127-10.12.255.255), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 2814749
76710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168558594, 168624127), F
ield 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [10] ecCount: 11, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655
-ff:ff:ff:ff:ff:ff), nw_src (168624128-10.13.0.0, 4294967295-255.255.255.255), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 281
474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168624128, 429496729
5), Field 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
faults size: 2
⚠ 0   Git Graph
```

原因分析

问题出在 VeriFlow 的 fault 管理逻辑：

```C++
for(unsigned int i = 0; i < faults.size(); i++) {
    if (packetClass.subsumes(faults[i])) {  // 完全包含才删除
        faults.erase(faults.begin() + i);
        i--;
    }
}
```

`subsumes` 要求新 EC 在每个字段上都**完全包含**旧 fault 才会删除。但 `fix_path.py` 产生的 EC 可能与旧 fault 只是**部分重叠**（有交集但不完全包含），导致旧 fault 残留。

修复方案

从旧 fault 中**精确扣除**与新 packetClass 的交集部分。扣完为空则移除 fault；扣完有剩余则保留剩余碎片。

添加 `intersects()` 方法

在 `EquivalenceClass.h` 中声明，在 `EquivalenceClass.cpp` 中实现：

```C++
bool EquivalenceClass::intersects(const EquivalenceClass &other) const
{
    for(int i = 0; i < ALL_FIELD_INDEX_END_MARKER; i++)
    {
        if(this->lowerBound[i] > other.upperBound[i] ||
           this->upperBound[i] < other.lowerBound[i])
            return false;  // 任一字段无重叠 → 不相交
    }
    return true;  // 所有字段都有重叠 → 相交
}
```

添加 `subtractIntersection()` 方法

在 `EquivalenceClass.h` 中声明：

```C++
vector<EquivalenceClass> subtractIntersection(const EquivalenceClass &other) const;
```

在 `EquivalenceClass.cpp` 中实现核心逻辑：

```C++
vector<EquivalenceClass> EquivalenceClass::subtractIntersection(const EquivalenceClass &other) const
{
    vector<EquivalenceClass> result;

    // 没有交集，返回自身不变
    if(!this->intersects(other)) {
        result.push_back(*this);
        return result;
    }

    // other 完全包含 this，扣完为空
    if(other.subsumes(*this)) {
        return result; // 空 vector → fault 被完全消除
    }

    // 逐维切割：维护"当前剩余"区域，逐字段切出不被 other 覆盖的边角
    EquivalenceClass current = *this;

    for(int i = 0; i < ALL_FIELD_INDEX_END_MARKER; i++) {
        if(other.lowerBound[i] ≤ current.lowerBound[i] &&
           other.upperBound[i] ≥ current.upperBound[i]) {
            continue; // 该字段被完全覆盖，跳过
```

```
        }

        // 左侧碎片: [cL, pL-1]
        if(current.lowerBound[i] < other.lowerBound[i]) {
            EquivalenceClass leftPart = current;
            leftPart.upperBound[i] = other.lowerBound[i] - 1;
            result.push_back(leftPart);
            current.lowerBound[i] = other.lowerBound[i];
        }

        // 右侧碎片: [pU+1, cU]
        if(current.upperBound[i] > other.upperBound[i]) {
            EquivalenceClass rightPart = current;
            rightPart.lowerBound[i] = other.upperBound[i] + 1;
            result.push_back(rightPart);
            current.upperBound[i] = other.upperBound[i];
        }
    }

    // 循环结束后 current 就是交集本身，丢弃
    return result;
}
```

替换 VeriFlow.cpp 中的判断逻辑

将所有原来的循环:

C++

```
for(unsigned int i = 0; i < faults.size(); i++) {
    if (packetClass.subsumes(faults[i])) {
        faults.erase(faults.begin() + i);
        i--;
    }
}
```

替换为:

C++

```
vector<EquivalenceClass> newFaults;
for(unsigned int i = 0; i < faults.size(); i++) {
    vector<EquivalenceClass> remaining = faults[i].subtractIntersection(packetClass);
    for(auto& r : remaining) {
        newFaults.push_back(r);
    }
}
faults = newFaults;
```

原理说明

逐维切割算法的工作方式是这样的：对每个字段，检查 fault 是否比 packetClass 范围更宽。如果是，就把多出来的左侧或右侧部分切下来作为独立碎片保留（这些部分还没被验证过），然后将"当前剩余"缩小到和 packetClass 的交集范围，继续处理下一个字段。最终剩下的就是交集本身（已被新规则覆盖验证过的部分），丢弃。

修复结果

开几个终端，执行以下命令:

bash

```
# terminal 1
TOPO=simple.txt CONFIG=simple.config.json uv run ryu-manager ryu.app.ofctl_rest as_switch.py --ofp-tcp-
listen-port 1024
# terminal 2
veriflow/VeriFlow 6633 127.0.0.1 1024 simple.txt veriflow.log
```

```
# terminal 3
sudo ./simple.py
mininet> illinois ping wisconsin
mininet> ucla1 ping illinois # 触发黑洞

# terminal 4
uv run ./fix_path.py
```

```
ziyu …/sdn-lab4 main !? lab4 v3.9.25 11:57
» sudo ./simple.py
usc1 usc1-eth0:s2-eth1
usc2 usc2-eth0:s7-eth1
wisconsin wisconsin-eth0:s4-eth1
*** Starting CLI:
mininet> illinois ping wisconsin
PING 10.10.0.2 (10.10.0.2) 56(84) bytes of data.
64 bytes from 10.10.0.2: icmp_seq=1 ttl=64 time=89.1 ms
64 bytes from 10.10.0.2: icmp_seq=2 ttl=64 time=14.7 ms
64 bytes from 10.10.0.2: icmp_seq=3 ttl=64 time=14.2 ms
64 bytes from 10.10.0.2: icmp_seq=4 ttl=64 time=14.1 ms
^C
--- 10.10.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 14.096/33.033/89.142/32.394 ms
mininet> ucla1 ping illinois
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=64 time=180 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=64 time=88.9 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=64 time=88.2 ms
64 bytes from 10.10.0.1: icmp_seq=4 ttl=64 time=88.3 ms
64 bytes from 10.10.0.1: icmp_seq=5 ttl=64 time=88.1 ms
^C
--- 10.10.0.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4005ms
rtt min/avg/max/mdev = 88.134/106.682/179.870/36.595 ms
mininet>
```

```
ziyu …/sdn-lab4 main !? lab4 v3.9.25 11:56
» veriflow/VeriFlow 6633 127.0.0.1 1024 simple.txt veriflow.log
[VeriFlow::verifyRule] [0] ecCount: 4, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427520-10.10.0.0, 168427520-10.10.0.0), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168427520, 168427520), Field 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [1] ecCount: 4, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427521-10.10.0.1, 168427521-10.10.0.1), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168427521, 168427521), Field 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [2] ecCount: 4, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427522-10.10.0.2, 168427522-10.10.0.2), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168427522, 168427522), Field 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [3] ecCount: 4, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427523-10.10.0.3, 168460287-10.10.127.255), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168427523, 168460287), Field 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
faults size: 8

[VeriFlow::verifyRule] ecCount: 1
[VeriFlow::verifyRule] [0] ecCount: 1, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427521-10.10.0.1, 168427521-10.10.0.1), nw_dst (168558593-10.12.0.1, 168558593-10.12.0.1), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168427521, 168427521), Field 9 (168558593, 168558593), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
faults size: 9
```

> fix之前

```
ziyu …/sdn-lab4 main !? lab4 v3.9.25 11:56
» veriflow/VeriFlow 6633 127.0.0.1 1024 simple.txt veriflow.log
[VeriFlow::verifyRule] [6] ecCount: 12, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168427523-10.10.0.3, 168460287-10.10.127.255), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168427523, 168460287), Field 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [7] ecCount: 12, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168460288-10.10.128.0, 168558591-10.11.255.255), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168460288, 168558591), Field 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [8] ecCount: 12, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168558592-10.12.0.0, 168558592-10.12.0.0), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168558592, 168558592), Field 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [9] ecCount: 12, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168558593-10.12.0.1, 168558593-10.12.0.1), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168558593, 168558593), Field 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [10] ecCount: 12, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168558594-10.12.0.2, 168624127-10.12.255.255), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168558594, 168624127), Field 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
[VeriFlow::verifyRule] [11] ecCount: 12, [EquivalenceClass] dl_src (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), dl_dst (0-00:00:00:00:00:00, 281474976710655-ff:ff:ff:ff:ff:ff), nw_src (168624128-10.13.0.0, 4294967295-255.255.255.255), nw_dst (168558592-10.12.0.0, 168624127-10.12.255.255), Field 0 (0, 65535), Field 1 (0, 281474976710655), Field 2 (0, 281474976710655), Field 3 (2048, 2048), Field 4 (0, 4095), Field 5 (0, 7), Field 6 (0, 1048575), Field 7 (0, 7), Field 8 (168624128, 4294967295), Field 9 (168558592, 168624127), Field 10 (0, 255), Field 11 (0, 63), Field 12 (0, 65535), Field 13 (0, 65535)
faults size: 0
```

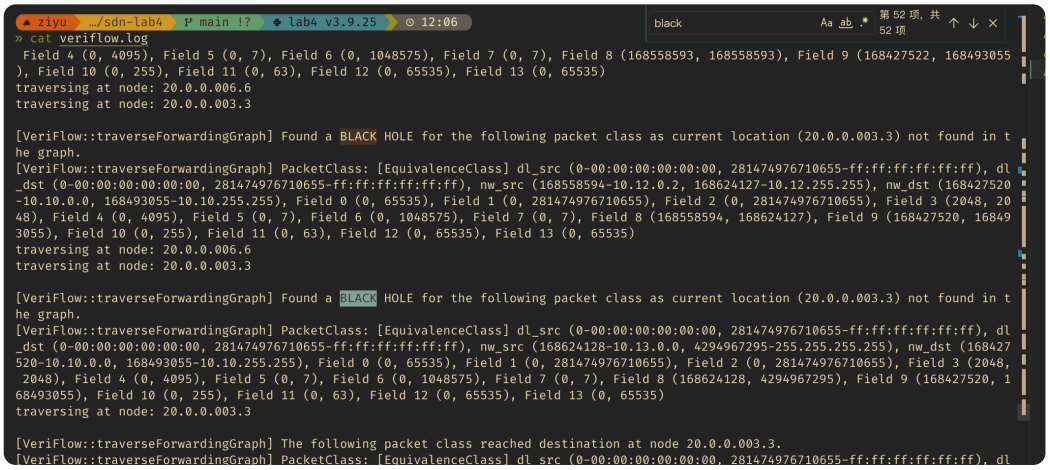
> fix之后

可以看到，最后`faults size`成功归零。

选做题

选做1：分析黑洞产生原因

目前，虽然通过修改fault判断逻辑，解决了错误的fault计数；但是还是没有真正解决黑洞的问题。可以看到目前还是会产生黑洞（下图）。



下面分析黑洞的原因和解决方法。

产生原因

问题出在 `as_switch.py` 的 `handle_ipv4` 函数中，有三个层面的问题：

问题 1：匹配粒度不一致

- **跨 AS 转发**（第 147、172 行）：使用**子网匹配**（`srcnet`, `dstnet`）

```python
out_port = add_path(route, None, None, srcnet, dstnet)
# 例如 srcnet="10.12.0.0/16", dstnet="10.10.0.0/16"
```

- **AS 内转发**（第 138 行）：使用**精确匹配**（`src_ip`, `dst_ip`）

```python
out_port = add_path(dpid_path, None, None, src_ip, dst_ip)
# 例如 src_ip="10.12.0.1", dst_ip="10.10.0.1"
```

子网规则把大量地址导向了只有精确规则的交换机，对于子网中不存在的地址会产生黑洞。

问题 2：路径分段安装（根本原因）

即使把匹配改为精确匹配，跨 AS 转发的路径是**分段安装**的：

1. 第一次 PacketIn：当前交换机只安装到 gateway 的路径
2. 数据包到了 gateway，触发第二次 PacketIn：gateway 安装到 peer 的路径
3. 数据包到了 peer，触发第三次 PacketIn：peer 安装到目的地的路径

VeriFlow 在第一段规则安装的瞬间就做验证——发现后续交换机还没有规则 → 报告黑洞。

问题 3：完整路径仍然可能被源端优先安装顺序打断

后来把跨 AS 路径一次性算成完整路径后，日志中仍然出现了黑洞。原因是 `add_path()` 虽然拿到了完整路径，但仍然按源端到目的端的顺序下发 FlowMod。以 `ucla1 → ucla2 → illinois` 为例：

1. 先安装 `ucla1 → ucla2`，VeriFlow 立即从 ucla1 遍历到 ucla2，但 ucla2 的规则还没安装，因此报告黑洞。
2. 再安装 `ucla2 → illinois`，VeriFlow 遍历到 illinois，但 illinois 到主机的规则还没安装，仍可能报告黑洞。
3. 最后目的端规则安装完成后，之前记录的 fault 才会被消除。

因此，完整路径本身还不够；必须让下游规则先存在，再让上游规则生效。

是否影响实际使用

不影响。实际数据包每到一个没有规则的交换机，都会触发 PacketIn（交换机发现没有匹配规则就上报控制器），控制器按需安装规则。最终整条路径上每个交换机都会有规则，ping 能正常通信。VeriFlow 只是在中间状态（路径还没完全安装）就做了验证。

修复方案

如果让源 AS 控制器直接计算 `当前交换机 → gateway → peer → 目的主机` 的完整路径，确实能避免瞬态黑洞，但这会破坏 AS 隔离：源 AS 等于知道了下游 AS 内部拓扑。因此采用是**分域计算 + 下游 ready 后源 AS 放行**的方案。

具体流程是：

1. 每个 AS 只计算自己域内的局部路径。
2. 源 AS 只知道本 AS 出口 gateway 和对端 peer，不直接计算 peer 后面的路径。
3. 源 AS 先请求下游 AS 为该 flow 准备路径。
4. 下游 AS 在本域内按下游到上游安装规则，并返回 ready。
5. 源 AS 收到 ready 后，才安装本 AS 到 peer 的规则并 PacketOut 当前数据包。

实验中仍然只有一个 `as_switch.py` 控制器进程，所以代码里用递归函数模拟“请求下游 AS 控制器准备路径”。调用者只拿到 ready 结果，不拼接、不读取下游 AS 的内部路径。

首先修改 `add_path()` 的流表下发顺序。新增辅助函数：

```python
def _downstream_first(port_path):
    return list(reversed(port_path))
```

然后把 `add_path()` 中发送 FlowMod 的循环改成：

```python
            # 按下游到上游的顺序安装，避免 VeriFlow 看到“上游已指向下游，
            # 但下游规则还没装好”的中间状态。
            for node in _downstream_first(port_path):
                waypoint_dpid, out_port = node
                send_flow_mod(waypoint_dpid, dl_src, dl_dst, nw_src, nw_dst, None, out_port, priority)
```

然后新增按目的地址选择路由前缀的辅助函数：

```python
def _prefix_length(network):
    return int(network.split("/")[1])


def _find_destination_network(gateways_cfg, switch_net, dst_ip):
    matched = []
    for dst_candidate in gateways_cfg.get(switch_net, {}):
        if utils.ipv4.in_net(dst_candidate, dst_ip):
            matched.append(dst_candidate)
    if not matched:
        return None
    return max(matched, key=_prefix_length)
```

最后，把 `as_switch.py` 的跨 AS 转发分支替换为分域 ready 逻辑：

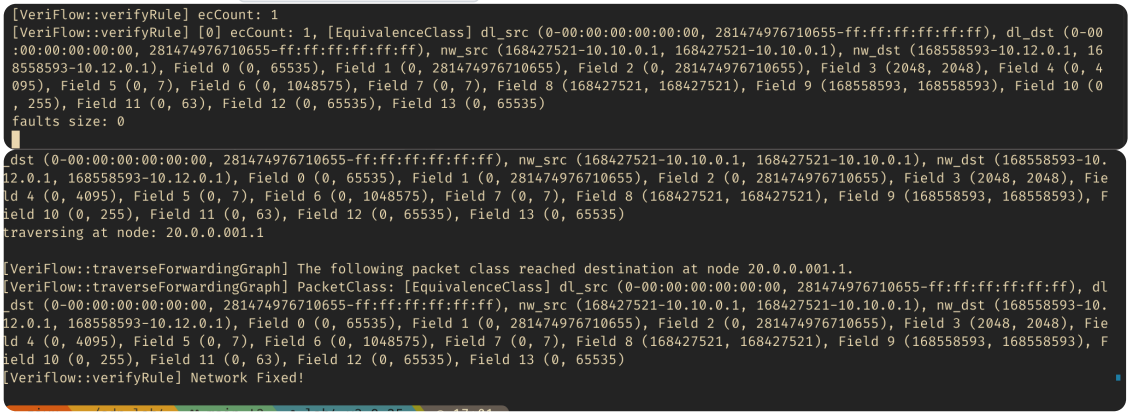
```python
        def prepare_as_path(entry_dpid, visited_nets):
            entry_net = self.routing_cfg["switch_nets"][entry_dpid]
            if entry_net in visited_nets:
                return None

            # 实验中仍是一个控制器进程；这里用递归调用模拟“请求 entry_dpid
            # 所在 AS 的控制器准备路径”。调用者只拿到 ready 结果，不读取
            # 下游 AS 的内部路径，从而保持 AS 间只通过 peer 暴露边界信息。
            next_visited_nets = visited_nets | {entry_net}
```

```
            if utils.ipv4.in_net(entry_net, dst_ip):
                # 目的主机在本 AS 内，由本 AS 自己计算并安装域内路径。
                local_path = self.network_awareness.shortest_path(entry_dpid, dst_ip)
                if not local_path:
                    return None
                return add_path(local_path, None, None, src_ip, dst_ip)

            # 本 AS 只根据自己的路由表选择出口 gateway 和对端 peer，
            # 不直接计算 peer 所在 AS 的内部路径。
            downstream_dstnet = _find_destination_network(
                self.routing_cfg["gateways"], entry_net, dst_ip)
            if downstream_dstnet is None:
                return None

            candidate_gateways = self.routing_cfg["gateways"][entry_net][downstream_dstnet]
            min_gw = closest_gateway(entry_dpid, candidate_gateways)
            if min_gw is None:
                return None

            peer = self.routing_cfg["peers"][str(min_gw)][downstream_dstnet]

            # 先等待下游 AS 准备好。只有 peer 后面的路径 ready 后，
            # 当前 AS 才安装 entry_dpid → min_gw → peer 这一段。
            if prepare_as_path(peer, next_visited_nets) is None:
                return None

            if entry_dpid == min_gw:
                route_to_peer = [min_gw, peer]
            else:
                path_to_gw = self.network_awareness.shortest_path(entry_dpid, min_gw)
                if not path_to_gw:
                    return None
                route_to_peer = path_to_gw + [peer]

            return add_path(route_to_peer, None, None, src_ip, dst_ip)

        out_port = prepare_as_path(dpid, set())
        if out_port is None:
            return
```

以 `ucla1 ping illinois` 为例，源 AS 只知道出口 `ucla1` 和 peer `ucla2`。它先请求 peer 所属 AS 准备路径；下游 AS 自己安装 `illinois → host`，再安装 `ucla2 → illinois`；下游返回 ready 后，源 AS 才安装 `ucla1 → ucla2`。这样既不会出现“包到了下游边缘交换机但规则还没安装”的黑洞，也避免源 AS 直接知道下游 AS 的内部路径。

修复结果

修改后重启实验，执行 `ucla1 ping illinois`，VeriFlow 不再报告 BLACK HOLE，faults size 最终归零。



实验总结

本实验通过 VeriFlow 网络验证工具，深入理解了 SDN 网络中转发**环路**和**黑洞**两种典型故障的检测与修复。在热身部分，通过修改 VeriFlow 源码实现了等价类数量统计、环路路径打印和 EC 五元组格式化输出，掌握了 VeriFlow 基于等价类划分和转发图遍历的核心验证原理。这个实验加深了我对 **SDN 规则安装**、**OpenFlow PacketIn/FlowMod 交互机制**以及**VeriFlow工具**的理解。